Web Application Development

Lab 04: Building Custom REST Endpoints with Spring Boot and MongoDB

Prerequisites

Before starting, ensure that:

- You have a running Spring Boot application connected to a local or cloud-hosted **MongoDB** instance.
- Your project contains a structured TemperatureLog model (Document), a corresponding TemperatureRepository interface, and a functioning TemperatureService.
- Your current database collection has a few generated test records containing mixed input units (e.g., both Celsius and Fahrenheit records).

Exercise 1: Adding Business Logic (The Safety Warning Endpoint)

Step 1: Implementing Business Logic in the Service Layer

Open your TemperatureService.java class and implement the computational safety logic. This ensures your controller does not contain hardcoded validation metrics.

```
// Add this method to TemperatureService.java

public String getSafetyWarning(double value, String unit) {
    String cleanUnit = unit.trim().toUpperCase();
    double celsiusTemp = value;

    // Convert to a standardized unit (Celsius) for uniform evaluation
    if ("FAHRENHEIT".equals(cleanUnit) || "F".equals(cleanUnit)) {
        celsiusTemp = (value - 32) * 5 / 9;
    }

    // Determine the environmental safety message
    if (celsiusTemp >= 38.0) {
        return "Warning: " + value + "°" + cleanUnit + " is dangerously HOT! Stay
hydrated.";
    } else if (celsiusTemp <= 0.0) {
        return "Warning: " + value + "°" + cleanUnit + " is freezing cold! Bundle
up.";
    } else {
        return "The temperature is comfortable and safe.";
    }
}
```

Step 2: Exposing the Endpoint in the Controller

Open your TemperatureController.java class and expose the service logic to the web using mapping and request parameter annotations.

```
@GetMapping("/safety-check")
public String checkTemperatureSafety(
    @RequestParam double value,
    @RequestParam String unit
) {
    return temperatureService.getSafetyWarning(value, unit);
}
```

Key Concept: The @RequestParam annotation binds query parameters from the request URL directly to the method arguments.

Step 3: Testing the Endpoint

Start your Spring Boot application and navigate to the following URLs in a web browser or Postman client:

Test Case A: Dangerous High Heat

- **URL:** http://localhost:8080/api/temperatures/safety-check?value=102&unit=F
- **Expected Response Body (Plain Text):**

Warning: 102.0°F is dangerously HOT! Stay hydrated.

Test Case B: Safe Temperature

- **URL:** http://localhost:8080/api/temperatures/safety-check?value=21&unit=C
- **Expected Response Body (Plain Text):**

Plaintext

The temperature is comfortable and safe.

Exercise 2: Querying the Database (The Filtered History Endpoint)

Task Description

You will design a database-filtering API endpoint that queries your MongoDB collection to retrieve and return only the logged records that match a specific input temperature unit (e.g., retrieving only "CELSIUS" records).

Step 1: Utilizing Spring Data Query Methods in the Repository

Spring Data simplifies query design. By declaring a method with a specific naming pattern in your interface, Spring Boot automatically constructs the database query.

Open TemperatureRepository.java and declare the custom query method:

```java
package com.example.tempconverter.repository;

import com.example.tempconverter.model.TemperatureLog;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface TemperatureRepository extends MongoRepository<TemperatureLog, String> {

    // Spring Boot auto-generates a case-insensitive match query against 'inputUnit'
    List<TemperatureLog> findByInputUnitIgnoreCase(String inputUnit);
}
```

Step 2: Adding the Database Query Call to the Service Layer

Open TemperatureService.java and add a gateway method to access your newly defined repository query.

```java
// Add this method to TemperatureService.java

public List<TemperatureLog> getLogsByUnit(String unit) {
    return temperatureRepository.findByInputUnitIgnoreCase(unit.trim());
}
```

Step 3: Creating the Filter Route in the Controller

Open TemperatureController.java and create a sub-route on /history/filter to accept the filter criteria.

```java
// Add this mapping to TemperatureController.java

@GetMapping("/history/filter")
public java.util.List<TemperatureLog> getFilteredLogs(@RequestParam String
unit) {
    return temperatureService.getLogsByUnit(unit);
}
```

Step 4: Testing Your MongoDB Query

Use Postman or your browser to query the database. Ensure you have submitted various temperature conversions beforehand to populate your database with diverse entries.

- **URL:** http://localhost:8080/api/temperatures/history/filter?unit=celsius
- **Expected Response Body (JSON Array):**


```json
[
  {
    "id": "65f3a1b2c3d4e5f67890ab11",
    "inputTemperature": 25.0,
    "inputUnit": "CELSIUS",
    "outputTemperature": 77.0,
    "outputUnit": "FAHRENHEIT",
    "timestamp": "2026-05-11T10:30:00.123"
  },
  {
    "id": "65f3a1b2c3d4e5f67890ab22",
    "inputTemperature": 0.0,
    "inputUnit": "celsius",
    "outputTemperature": 32.0,
    "outputUnit": "FAHRENHEIT",
    "timestamp": "2026-05-11T10:32:15.456"
  }
]
```

Discussion & Self-Study Questions

1. **Extending Spring Data Query Methods:** How would you modify your TemperatureRepository interface and your endpoint syntax to filter and return logs that are strictly greater than a user-specified input temperature threshold (e.g., retrieving all records where inputTemperature > 30)?

(Hint: Research Spring Data keywords like GreaterThan).